

ENGINEERING POST-MORTEM REPORT

CI/CD Pipeline Resolution & Architecture Documentation

Project Name:	Project Beta (Asset Register)	Date resolved:	May 21, 2026
Lead Engineer:	Satnam Singh	Deployment Environment:	Azure Static Web Apps (SWA)
CI/CD Platform:	GitHub Actions	Status:	SUCCESS (Pipeline Verified Green)

1. Executive Summary

This document serves as an engineering post-mortem and deployment chronicle for the automated release architecture of **Project Beta**. Over a sequence of iterations, the continuous integration and continuous deployment (CI/CD) engine encountered multi-layered configuration bottlenecks across directory paths, repository mapping, portal-forced overrides, and script mismatches.

Through systematic troubleshooting, isolation of monorepo dependencies, and manual compilation target routing, the pipeline was rebuilt from scratch. The final deployment architecture cleanly isolates automated native asset building inside GitHub containers and pipes static structures directly to Azure's Content Delivery Network (CDN), completely cutting out unstable auto-detection layers. The deployment workflow is now fully operational and optimized for hands-free future iterations.

2. Incident Chronology & Root Cause Analysis

During the wiring phase of the repository automation, three core architectural problems collided, causing persistent pipeline failures (`Process completed with exit code 1`). Below is the analytical structural breakdown of these bottlenecks:

Encountered Error & Symptom	Root Cause Isolation	Engineering Solution Applied
Input required and not supplied: username Concurrent phantom workflow failures in the action UI dashboard.	Ghost Configuration Overlap: The presence of an obsolete file <code>deploy.yml</code> inside the directory was tracking a legacy Docker / Azure Container Registry framework that was missing structural access tokens.	Conducted a local terminal diagnostic (<code>ls .github/workflows/</code>) to catch phantom references and performed a destructive removal of the file from the remote codebase.
npm error Missing script: "build" Runner crashes instantly during dependency validation.	Monorepo Blindspot: The repository was structured as a multi-tier project with nested blocks (<code>beta-frontend/</code> and <code>beta-backend/</code>). The runner executed at the absolute root layer, identifying a template file instead of the actual React/Vite installation.	Injected a localized path instruction (<code>working-directory: "./beta-frontend"</code>) to drop the node process into the explicit folder containing Vite config files.
Artifact folder: 'vite' not found Vite compiles smoothly to <code>dist/</code> but Azure fails the deployment step.	Portal Environment Lock-In: During the provisioning phase of the Azure SWA instance, the cloud provider hardcoded a local constraint expecting an output directory named <code>vite</code> , overriding standard <code>dist</code> compilation settings.	Bypassed Microsoft Oryx builds natively (<code>skip_app_build: true</code>), and utilized standard bash utilities to forcefully translate folders (<code>mv dist vite</code>) prior to the Azure upload hand-off.

💡 Key Takeaway on Architectural Decoupling:

Relying entirely on cloud providers' auto-detection systems creates implicit dependencies. By handling compilation step natively within GitHub Actions and providing explicit folder footprints, we guarantee cross-platform predictability.

3. Automated Production Blueprint (Final YAML Configuration)

The configuration file below represents the unified, verified architecture residing inside `.github/workflows/azure-static-web-apps.yml`. It securely decouples the compilation from the cloud provider, processes dependencies, translates monorepo constraints, and fulfills Azure's hardcoded artifact requirements.

```

name: Azure Static Web Apps CI/CD

on:
  push:
    branches:
      - main
  pull_request:
    types: [opened, synchronize, reopened, closed]
    branches:
      - main

jobs:
  build_and_deploy_job:
    if: github.event_name == 'push' || (github.event_name == 'pull_request' && github.event_action != 'close')
    runs-on: ubuntu-latest
    name: Build and Deploy Job
    steps:
      - uses: actions/checkout@v3
        with:
          submodules: true

      - name: Setup Node.js
        uses: actions/setup-node@v3
        with:
          node-version: '20'

      # Step 1: Native Compiling directly inside Monorepo Folder
      - name: Install Dependencies & Build Frontend
        run: |
          npm install
          npm run build
          mv dist ../vite
        working-directory: "../beta-frontend"

      # Step 2: Push Pre-Compiled Independent Footprint up to Azure
      - name: Deploy to Azure SWA
        id: builddeploy
        uses: Azure/static-web-apps-deploy@v1
        with:
          azure_static_web_apps_api_token: ${ secrets.AZURE_STATIC_WEB_APPS_API_TOKEN }
          repo_token: ${ secrets.GITHUB_TOKEN }
          action: "upload"
          ##### Production App Configurations #####
          app_location: "/beta-frontend"
          api_location: ""
          output_location: "vite"
          skip_app_build: true # Forces Azure to accept pre-built static files
          ##### End of App Configurations #####

  close_pull_request_job:
    if: github.event_name == 'pull_request' && github.event_action == 'closed'
    runs-on: ubuntu-latest

```

```
name: Close Pull Request Job
steps:
  - name: Close Pull Request
    id: closepullrequest
    uses: Azure/static-web-apps-deploy@v1
    with:
      azure_static_web_apps_api_token: ${ secrets.AZURE_STATIC_WEB_APPS_API_TOKEN }
      action: "close"
```

4. Post-Resolution Operations Guide

With the green state confirmed, the deployment engine is entirely hands-free. To push updates or maintain the environment, follow these standard operating workflows:

4.1 Code Modification & Deployment Cycles

To push updates to production, execute standard Git version control sequences from your terminal root folder. The infrastructure handles directory targeting automatically:

```
git add .
git commit -m "feat: core UI visual refresh"
git push origin main
```

4.2 Secret Token Management

The raw deployment token is securely isolated. If the authentication block ever requires a roll or regeneration, do **not** touch the YAML file code. Instead, navigate through the UI path:

[GitHub Web Interface](#) → [Settings](#) → [Secrets and variables](#) → [Actions](#) → [Repository Secrets](#)

Update the variable value for `AZURE_STATIC_WEB_APPS_API_TOKEN` here to preserve continuous integration handshakes seamlessly.

System Verification Status: Green

The frontend application is officially live, globally distributed on the CDN, and tied to your repository commit hashes. No further pipeline adjustments are required for the static web app container layer.